

HLLM: SINGLE PASS DECODING FOR GENERATIVE RERANKING

Emil Laftchiev, Prachi Agrawal, Moe Kayali, & Bixing Yan

Meta Platforms, Inc.

{emillaftchiev, praa, kayali, bixing}@meta.com

Qi Xu, Zijie Lei, Chen Qiu, Zhi Hua, Ke Li & Luke Simon

Meta Platforms, Inc.

ABSTRACT

Large language models (LLMs) achieve state-of-the-art generative ranking quality, but the ranking they produce must be *decoded*, and autoregressive decoding spends one sequential forward pass per emitted token. We observe that the only tokens a ranker must emit are the N ordinal values naming the items in ranked order, and that this narrow, permutation-structured output format admits decoding strategies which are much more efficient than left-to-right generation. We introduce HLLM (Hungarian LLM), a format-specialized decoding strategy that decodes all N ordinals in $O(1)$ forward passes. HLLM reads an $N \times K$ item–position score matrix off the LLM’s prefill hidden states with a lightweight self-attention head, then decodes the ordinals as the optimal bipartite assignment of that matrix via the Hungarian algorithm, yielding a valid permutation by construction rather than by repair. Through a systematic study of training signals and backbone adaptation, we show that LoRA-based fine-tuning combined with teacher ranking distillation reaches **28ms** end-to-end inference, a speed-up of $64\times$ while maintaining ranking quality on par with the teacher. We provide a complete ablation decomposing the contributions of architecture, training signal, and backbone adaptation. Our framework connects generative ranking to combinatorial optimization, opening a path toward other $O(1)$ -decode mechanisms for real-time ranking.

1 INTRODUCTION

Generative ranking, producing a complete ordering of N candidates jointly rather than scoring each independently, is critical for recommendation, advertising, and search. Autoregressive LLMs have recently emerged as powerful generative rankers, achieving superior quality by jointly reasoning over all candidates in context (Sun et al., 2023). The output of such a ranker is a permutation, and the model must *emit* that permutation as tokens: concretely, the N ordinal values that name the items in ranked order. Ranking is therefore an unusually *format-specialized* generation problem—the output alphabet is $\{1, \dots, N\}$, the output length is known before decoding begins, and every value must appear exactly once. It is this structure that determines which decoding strategies are available, and it is the axis along which we organize both the prior work and our contribution.

Autoregressive decoding, however, spends one sequential forward pass over the KV cache per emitted token, so decoding the N ordinals uses $O(N \cdot T)$ passes, where T is the number of tokens per item identifier. This is a bottleneck of the *decoder*, not of the model: the prefill that computes the candidate representations is already parallel and already optimized in its computational complexity. As an example, in the experiments for this paper, a teacher with an explicit reasoning trace reaches 1807 ms per request on an A100 GPU, of which 28 ms is prefill. We address the *re-ranking* stage, in which a candidate set of N items retrieved by an upstream system must be ordered by relevance under strict latency constraints.

Decoding strategies. Because the ordinals are what must be decoded, the design space is a space of *decoders* over a fixed backbone. Autoregressive decoding emits them left to right, one forward

pass per token: exact and requiring no architectural change, but with a pass count that grows with the slate. Parallel decoders—non-autoregressive rerankers and multi-token predictors (Ren et al., 2024), and diffusion-based rerankers (Liu et al., 2026)—emit all positions at once and refine them over several steps, reducing the pass count but not to one, and without guaranteeing that the emitted ordinals form a permutation. We ask whether the ordinals can be decoded in a single pass:

Can a ranker decode all N ordinals in $O(1)$ forward passes and still recover the ranking that an autoregressive LLM computes over hundreds of sequential passes?

We answer yes, with HLLM, which exploits a connection between ranking and optimal assignment. The key insight is that an LLM’s prefill hidden states, computed in a single parallel forward pass over all candidates, already encode the comparative information that the ordinals express, via the backbone’s self-attention layers. Autoregressive decoding re-serializes that information one token at a time; HLLM instead decodes it in place. A lightweight self-attention head turns the prefill states into an $N \times K$ item–position score matrix, and the Hungarian algorithm (Kuhn, 1955) decodes all N ordinals from it as the optimal bipartite assignment, with a valid permutation guaranteed by construction.

Contributions.

1. **A formulation of ordinal decoding as optimal assignment on hidden states.** We cast the emission of the N item ordinals as maximum-weight bipartite matching over an $N \times K$ item–position score matrix whose weights are read directly off the model’s hidden states rather than off the output logits. This is the formulation that makes exact combinatorial decoding possible: the Hungarian algorithm recovers the optimal permutation at inference, from one pass. Output validity is guaranteed by construction rather than learned.
2. **HLLM, an $O(1)$ -pass decoding strategy for generative ranking.** A single prefill over all candidates yields per-item hidden states; a lightweight self-attention head maps them to the score matrix; the Hungarian solver decodes the N ordinals from it. Decoding therefore takes a constant number of forward passes, independent of N , removing the sequential decode dependency that dominates autoregressive ranking latency.
3. **An empirical study on proprietary data and an open dataset.** We evaluate the performance and latency of the approach on a proprietary dataset, finding that HLLM achieves a speed-up of $64\times$ over an autoregressive LLM while maintaining ranking performance. We also validate our approach on an open-source ranking dataset, Amazon Beauty—and find similar speedups. We further ablate the experiments across three scoring-head configurations. We also isolate the contributions of the backbone and the self-attention layers.

2 BACKGROUND AND RELATED WORK

2.1 LLM-BASED RANKING AND GENERATIVE RETRIEVAL

Sun et al. (2023) introduced RankGPT, showing that LLMs can rerank generatively, decoding permutations autoregressively with a sliding window. Tay et al. (2022) pioneered generative retrieval with the Differentiable Search Index, encoding document identifiers directly in model parameters, and De Cao et al. (2021) showed autoregressive entity generation is viable for structured retrieval. Most relevant to our work, Reddy et al. (2024) proposed FIRST, which uses first-decoded-token logits for ranking. This is the closest prior approach to constant-pass decoding, but it reads the ordering out of the output token distribution rather than out of the hidden states. HLLM keeps the same output as all of these—the item ordinals—and changes only how they are decoded.

2.2 DECODING STRATEGIES FOR GENERATIVE RANKING

Beyond autoregressive decoding, non-autoregressive (NAR) methods for machine translation established that parallel generation can achieve substantial speedups at some degradation of quality. Gu et al. (2018) introduced NAR translation via fertility prediction; Ghazvininejad et al. (2019) recovered quality through iterative refinement of masked positions; and Qian et al. (2021) introduced curriculum-based NAR training. NAR methods for general text generation typically suffer

more than 10% quality degradation, and they still take several refinement passes per output. HLLM sidesteps both by exploiting the permutation structure of ranking: rather than decoding arbitrary text in parallel, we decode the ordinals as the solution of a constrained combinatorial problem that admits an exact polynomial-time solution, in a constant number of passes.

2.3 OPTIMAL TRANSPORT AND DIFFERENTIABLE PERMUTATION LEARNING

Our training procedure builds on differentiable relaxations of discrete permutations. Cuturi (2013) introduced entropic regularization for efficient optimal transport, providing the algorithmic foundation for the Sinkhorn operator, and Mena et al. (2018) extended this to learning latent permutations with Gumbel-Sinkhorn networks. Alternative differentiable sorting approaches include SoftSort (Prillo & Eisenschlos, 2020) and differentiable sorting networks (Petersen et al., 2021). Our contribution is connecting these permutation-learning tools to LLM hidden states.

2.4 KNOWLEDGE DISTILLATION FOR RANKING

Hinton et al. (2015) introduced soft-target distillation. For ranking, Hofstätter et al. (2020) demonstrated cross-architecture distillation from cross-encoders to bi-encoders, and Pradeep et al. (2023) distilled generative reranking ability into smaller models. We differ in two ways: we distill full permutations via a Sinkhorn cross-entropy objective, and our ablation reveals a previously undocumented interaction between signal richness and backbone capacity.

2.5 HUNGARIAN ASSIGNMENT IN DEEP LEARNING

DETR (Carion et al., 2020) uses bipartite matching between predicted and ground-truth detections for set-based loss computation. Berthet et al. (2020) provided theoretical foundations for learning with differentiable perturbed combinatorial optimizers. In mechanism design, Dütting et al. (2019) learn optimal multi-item auctions via neural networks, and for ad allocation Zheng et al. (2025) introduced non-autoregressive generative auctions with position-query cross-attention. Critically, we use Hungarian matching *not* for loss computation as in DETR, but as the inference-time decoder that replaces autoregressive generation. Similarly, DiffuRank (Liu et al., 2026) applies the Hungarian algorithm to the decoded output of the LLM. We instead apply it to the hidden states, as the decoder itself, so the permutation constraint is enforced during decoding rather than restored afterwards.

2.6 EFFICIENT LLM INFERENCE

Speculative decoding (Leviathan et al., 2023) uses a small draft model verified by the target LLM for lossless speedup. Medusa (Cai et al., 2024) and EAGLE (Li et al., 2024) extend this with multiple parallel decoding heads, and LayerSkip (Elhoushi et al., 2024) enables early-exit inference. These methods all reduce the computational overhead of autoregressive decoding while preserving its left-to-right form; the pass count still scales with the output. HLLM is orthogonal and composable: it changes the decoding strategy itself, so the N ordinals emerge from a single pass.

Despite this breadth of prior work, no existing method decodes a complete ranking from an LLM backbone in a constant number of forward passes, at real-time latency, with near-lossless quality and a permutation guaranteed by construction.

3 PROBLEM FORMULATION

3.1 NOTATION AND SETUP

Let u denote a user context and $C = \{c_1, \dots, c_N\}$ a slate of N candidate items retrieved by an upstream system. The re-ranking task is to produce a permutation $\pi \in \Pi_N$ that orders items by relevance. An autoregressive teacher generates π via T sequential decode passes, incurring $O(N \cdot T)$ latency. We seek a student f_θ that decodes the same N ordinals, and hence the same π , in $O(1)$ decode passes—i.e. a single forward pass.

Latency decomposition. For an autoregressive ranker, inference computation is dominated by the decode phase: each token requires a full forward pass over the KV-cache. The following examples are from the experiments in this paper. With reasoning enabled $T \approx 657$ tokens; without reasoning $T \approx 39$. In both cases the prefill phase, which processes all N candidates in parallel, takes a fixed $t_{\text{prefill}} \approx 28$ ms. Our key observation: if the ranking can be extracted from the prefill hidden states alone, the $O(T)$ *sequential* decode computational complexity collapses to $O(1)$ —the ordinals are still decoded, but not one at a time.

3.2 DECODING A RANKING AS OPTIMAL ASSIGNMENT

Define a bipartite graph $G = (I \cup J, E)$ where $I = \{1, \dots, N\}$ indexes items and $J = \{1, \dots, K\}$ indexes rank positions. Let $\mathbf{M} \in \mathbb{R}^{N \times K}$ be a score matrix in which $\mathbf{M}_{ij}$ is the affinity of item i for rank position j . The optimal ranking is the maximum-weight bipartite matching

$$\pi^* = \arg \max_{\pi \in \Pi_N} \sum_{i=1}^N \mathbf{M}_{i, \pi(i)}. \quad (1)$$

Equation (1) is the linear assignment problem (LAP), solvable exactly in $O(N^3)$ by the Hungarian algorithm (Kuhn, 1955), or by the Jonker–Volgenant variant (LAPJV) (Jonker & Volgenant, 1987) which achieves $O(N^2)$ average-case complexity via shortest augmenting paths.

Complexity comparison. For $N = 50$, the autoregressive teacher requires $T = 657$ sequential forward passes. The Hungarian solver requires $O(N^3) \approx 1.25 \times 10^5$ arithmetic operations, roughly $8 \mu\text{s}$ on CPU. The full HLLM pipeline (one prefill plus one Hungarian solve, i.e. $O(1)$ passes for all N ordinals) is therefore bounded by the computational complexity of prefill, making the combinatorial optimization effectively free (Section 5.5).

3.3 DIFFERENTIABLE RELAXATION VIA SINKHORN

The Hungarian algorithm is non-differentiable: it returns a discrete permutation. For gradient-based training we require a continuous relaxation. We adopt the Sinkhorn operator (Cuturi, 2013; Mena et al., 2018), which projects a non-negative matrix onto the Birkhoff polytope of doubly-stochastic matrices via alternating row and column normalization:

$$\mathbf{S}^{(0)} = \exp(\mathbf{M}/\tau), \quad \mathbf{S}^{(l+1)} = \mathbf{D}_r^{-1} \mathbf{D}_c^{-1} \mathbf{S}^{(l)}, \quad (2)$$

where $\mathbf{D}_r$ and $\mathbf{D}_c$ are diagonal normalization matrices enforcing unit row and column sums, and $\tau > 0$ is a temperature controlling relaxation sharpness ($\tau \rightarrow 0$ recovers a permutation matrix).

Training objective. Given a teacher permutation matrix $\mathbf{P}^{\text{teacher}} \in \{0, 1\}^{N \times K}$, the Sinkhorn cross-entropy loss is

$$\mathcal{L} = - \sum_{i,j} \mathbf{P}_{ij}^{\text{teacher}} \log \mathbf{S}_{ij}^{(L)}. \quad (3)$$

This provides smooth gradients to $\mathbf{M}_\theta$ while driving the student’s output toward a valid permutation. At inference we bypass Sinkhorn entirely and apply the Hungarian algorithm to $\mathbf{M}$ directly, guaranteeing an exact permutation.

4 METHOD: HLLM

4.1 ARCHITECTURE OVERVIEW

HLLM comprises three components applied in sequence (Figure 1). Figure 1 depicts the winning configuration (self-attention head, $L = 2$, LoRA-adapted backbone); alternative head variants are ablated in Section 5.4.

Backbone (prefill-only, LoRA-adapted). The backbone is a 0.6B-parameter decoder-only transformer (28 layers, hidden dimension 1024, 16 attention heads) pretrained for language modeling and fine-tuned for ranking. It processes the full prompt containing the user context and all N candidates in a single forward pass. We gather the hidden state at each candidate’s readout position (the final token of its description), yielding $\{\mathbf{h}_1, \dots, \mathbf{h}_N\} \in \mathbb{R}^{N \times D}$.

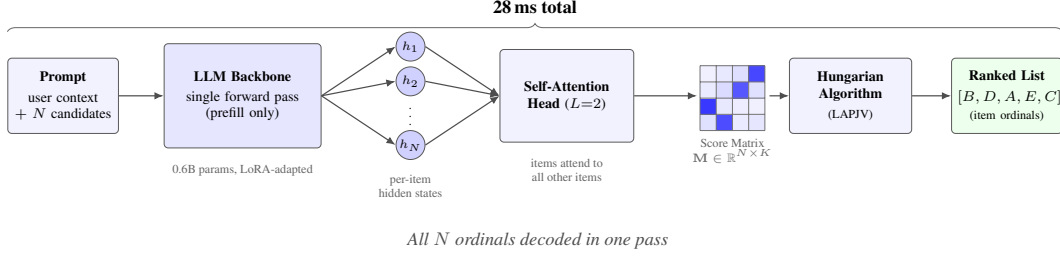


Figure 1: HLLM architecture. A single prefill pass over all N candidates yields per-item hidden states, which the self-attention head maps to an $N \times K$ score matrix. The Hungarian algorithm decodes all N item ordinals from that matrix in one shot: $O(1)$ forward passes, no sequential decode. End-to-end latency is 28 ms.

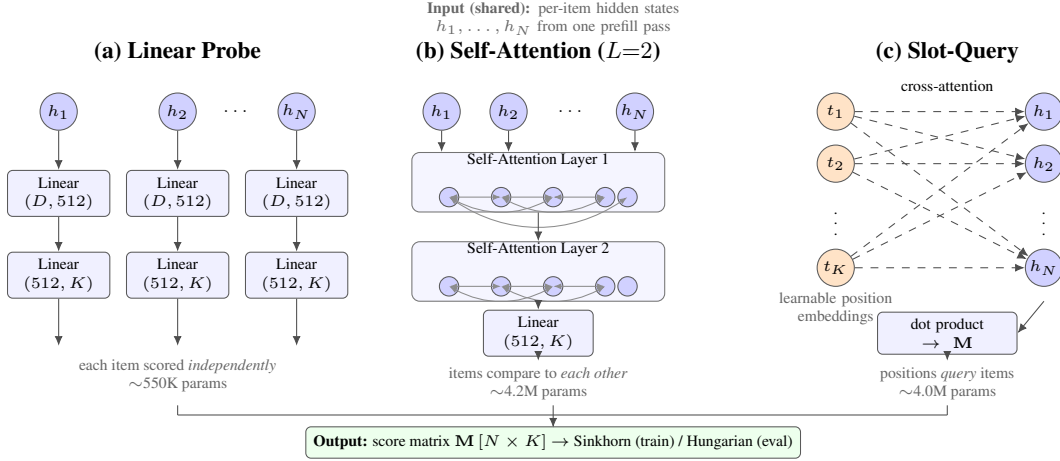


Figure 2: Scoring head variants. (a) A linear probe scores each item independently; all cross-item context arises implicitly from the backbone’s self-attention during prefill. (b) The self-attention head performs explicit item-to-item comparison before scoring. (c) The slot-query head inverts the formulation: K learnable position embeddings cross-attend over items. On a frozen backbone (a) is competitive; once the backbone is LoRA-adapted, (b) is strongest.

Scoring head (self-attention, $L = 2$). A lightweight transformer encoder with $L = 2$ self-attention layers operates over the N candidate vectors, allowing each item to attend to all others and thereby perform explicit pairwise comparison. A linear projection maps each refined representation to K rank-position scores, producing $\mathbf{M} \in \mathbb{R}^{N \times K}$.

Assignment decoder (Hungarian). Unlike DETR (Carion et al., 2020), which uses Hungarian matching during training for loss computation, we apply the Hungarian algorithm at inference as the decoding mechanism itself. Applied to $\mathbf{M}$ it yields the optimal one-to-one assignment: a valid permutation decoded in one shot. The Hungarian solver is thus HLLM’s decoder in the same sense that a sampling loop is an autoregressive decoder—it is the component that turns model outputs into the N emitted ordinals. During training, Equation (3) supplies gradients.

4.2 SCORING HEAD VARIANTS

We study three head architectures to isolate the contribution of cross-item comparison (Figure 2, Table 1).

Head	Key property	Params
Linear probe	Each item scored independently; all cross-item context arises implicitly from backbone self-attention	550K
Self-attention ($L=2$)	Explicit pairwise comparison: each item attends to all others before scoring	4.2M
Slot-query	Dual formulation: K learnable position embeddings cross-attend over items (“what fills rank 1?”)	4.0M

Table 1: Scoring head variants. All operate on identical frozen or LoRA-adapted backbone hidden states.

4.3 TRAINING: OFFLINE TEACHER RANKING DISTILLATION

Training proceeds in two phases.

Phase 1 (offline, one-time). The teacher, a well-trained autoregressive generative ranking model, decodes a ranking for every training slate via greedy decoding in direct-output mode (no chain-of-thought). These target permutations are computed once and stored.

Phase 2 (student training). The student minimizes Equation (3) against the pre-computed teacher permutations. There is no online interaction between teacher and student; the labels are fixed. This follows from the decoding strategy: HLLM’s decoder has no intermediate state and no rollout— $\mathbf{M}$ is conditioned only on the prompt, and the Hungarian solve is a deterministic function of $\mathbf{M}$ —so the train-time and inference-time input distributions are identical by construction. There is no exposure bias for on-policy distillation to correct and no student trajectory for the teacher to score, and Equation (3) is already a whole-permutation objective rather than a per-token surrogate. One offline teacher pass over the training set therefore suffices.

4.4 BACKBONE ADAPTATION VIA LoRA

We apply Low-Rank Adaptation (LoRA, Hu et al., 2022) with rank 64 to all linear projections in the backbone. This allows the backbone to reshape its hidden states to encode ranking-relevant information while preserving pretrained knowledge. As Section 5.3 shows, this adaptation is the enabler: without it, even rich teacher signals cannot be absorbed by the scoring head.

5 EXPERIMENTS

5.1 SETUP

Internal dataset. A proprietary re-ranking dataset ($\approx 189\text{K}$ training slates, 236K test requests), with up to 150 items per slate. **Teacher:** a well-trained autoregressive generative ranking model (0.6B-parameter decoder-only transformer, fine-tuned for ranking), evaluated both with and without an explicit reasoning trace.

Amazon Beauty dataset. The “beauty” products subset of the 2014 Amazon product dataset (He & McAuley, 2016). Slates comprise 50 candidate items to rank, with 21,245 training and 1,118 evaluation slates. **Teacher:** 32B Qwen model used for ranking. We also test a 0.6B Qwen model for comparison.

Student. The same 0.6B architecture, adapted via LoRA ($r = 64$). Trained via SFT from the respective teacher for each dataset.

Metrics. AUC, Recall@{1, 10}, NDCG@1. **Hardware.** NVIDIA A100 80GB; LAPJV solver on CPU.

5.2 MAIN RESULTS

We present the main results and a series of ablations below, note that ablations were performed on successive datasets. Table 2 compares HLLM against both teacher configurations. Removing the

Model	Latency	Speed-up	Decode passes	AUC	R@1	R@10	NDCG@1
Internal Dataset							
Teacher (with reasoning)	1807 ms	1.0×	768	0.5911	0.1634	0.7877	0.1776
Teacher (no reasoning)	88 ms	20.5×	39	0.5912	0.1635	0.7892	0.1779
HLLM (ours)	28 ms	64.5×	1	0.5907	0.1652	0.7912	0.1791
Amazon Beauty Dataset							
Qwen3-32B	5075 ms	1.0×	78	0.6292	0.1384	0.4232	0.1384
Qwen3-0.6B	921 ms	5.5×	26	0.5122	0.0232	0.2062	0.0179
HLLM 0.6B (ours)	113 ms	44.9×	1	0.6168	0.1321	0.4268	0.1347

Table 2: Main results. Every row decodes the same output: the N item ordinals naming the ranked slate. **Decode passes** counts the sequential forward passes required to do so; for autoregressive teachers under greedy decoding this equals the number of emitted tokens, while HLLM decodes the ordinals out of the prefill pass itself, so its count is constant in N . Speed-up is relative to the teacher within each dataset.

Track	Training signal	Backbone	AUC
1	Click labels (binary)	Frozen	0.5187
2	Teacher ranking (full permutation)	Frozen	0.5573
3	Teacher ranking (full permutation)	LoRA ($r=64$)	0.5842

Table 3: Signal richness interacts with backbone capacity. Rich permutation targets *hurt* a frozen backbone (Track 2 < Track 1) but become the strongest signal once the backbone can adapt (Track 3).

reasoning trace reduces teacher latency from 1807 ms to 88 ms, a $20\times$ reduction in decode passes, at essentially no reduction in quality, confirming that reasoning tokens are not load-bearing for ranking. HLLM decodes the same ordinals in a single pass, reaching 28 ms: $64\times$ faster than the reasoning teacher and $3.1\times$ faster than the no-reasoning teacher. Note what the second comparison isolates. The no-reasoning teacher has already stripped every non-essential token and decodes only the ranking itself; the remaining $3.1\times$ is attributable purely to the decoding strategy, not to shorter output. Quality is lossless on list-level metrics: HLLM is indistinguishable from both teachers on NDCG@1 (0.1791 HLLM vs. 0.1776 / 0.1779) and Recall@1 (0.1652 HLLM vs. 0.1634 / 0.1635). AUC is the metric where HLLM retains 99.9% of the teacher’s quality. We attribute the list-level gains to two factors: the Hungarian assignment enforces a globally consistent one-to-one ordering that autoregressive decoding only approximates, and distillation across the full training set averages out the teacher’s sampling noise (Section 6.1).

5.3 ABLATION: TRAINING SIGNAL $\times$ BACKBONE ADAPTATION

Table 3 isolates the interaction between signal richness and backbone adaptability. We find that freezing the backbone causes a significant penalty in ranking performance (row 2). Similarly, training on click labels only is not sufficient: the poor ranking performance indicates that the click-signal is too sparse a signal for good learning (row 1). Neither ingredient suffices alone; only their combination (Track 3) succeeds.

5.4 ABLATION: HEAD ARCHITECTURE

Table 4 shows our ablations on the three different head architectures described. The self-attention architecture described above performs best. Removing the self-attention layers and replacing them with a linear probe reduces the ranking performance, as does adding learned position-dependent embeddings. On the backbone that is LoRA-adapted, the self-attention head is the strongest configuration, consistent with our interpretation in Section 6.1.

Head	AUC	Recall @ 1
Self-attention, $L=2$ (HLLM)	0.5911	0.1652
Linear probe	0.5841	0.1650
Slot-query	0.5816	0.1579

Table 4: Ablation on the different scoring head architectures.

Component	Latency
Backbone prefill, all items (GPU)	27.9 ms
Scoring head, $L=2$ (GPU)	< 0.1 ms
LAPJV assignment, $N=50$ (CPU)	0.008 ms
Total	28 ms

Table 5: End-to-end latency decomposition. The combinatorial solver contributes under 0.03% of total latency.

5.5 SOLVER OVERHEAD

Table 5 decomposes end-to-end latency. The backbone prefill accounts for essentially all of it; the scoring head adds under 0.1 ms and the LAPJV solver 0.008 ms, under 0.03% of the total. This is the central efficiency argument: although the linear assignment problem is $O(N^3)$ in the worst case, at slate sizes of $N \leq 50$ it resolves in microseconds on CPU, fully overlapped by the GPU forward pass. The apparent overhead of introducing a combinatorial solver is therefore illusory. More importantly, the decomposition shows that HLLM has moved the bottleneck from a *sequential* dependency (autoregressive decode, which cannot be parallelized) to a *parallel* one (a single prefill over all candidates). Decoding all N ordinals has an impact of 0.008 ms of that budget. Any further latency reduction must come from the backbone itself, via pruning, quantization, or early exit, rather than from the ranking machinery, which is already effectively free.

6 ANALYSIS

6.1 THE CAPACITY GAP

Reproducing a full N -item permutation requires hidden states that encode fine-grained relative preferences between every pair of items. Click labels are binary and supply a simple gradient (“push clicked items up”). Teacher rankings demand that the score matrix reproduce exact orderings, which requires the backbone to encode pairwise preferences absent from its language-modeling-optimized representations. LoRA closes this capacity gap with roughly 4M trainable parameters on a 600M backbone. This explains the counterintuitive ordering in Table 3: a frozen backbone cannot reshape its representations to serve the richer target, so the richer signal becomes harmful rather than helpful.

6.2 STRUCTURAL GUARANTEES

By construction HLLM yields (i) 100% valid outputs, since the Hungarian algorithm always returns a valid permutation—the constraint is discharged inside the decoder rather than repaired after it; (ii) no degenerate solutions, since the one-to-one constraint forbids assigning multiple items to the same position; and (iii) deterministic inference, with no sampling temperature or beam-search artifacts. These properties are guaranteed rather than learned, and they account for part of the list-level advantage over the autoregressive teacher, which can and does emit invalid or repeated orderings.

7 CONCLUSION

We presented HLLM, a format-specialized decoding strategy that decodes a complete ranking—all N item ordinals—in $O(1)$ forward passes, at 28 ms end to end. We formalize the use of the Hungarian algorithm to decode a ranking directly from hidden states. We empirically validate this approach

on a proprietary dataset and an open-source benchmark, finding near-lossless ranking quality compared to autoregressive LLMs, at a 45–64 $\times$ inference speedup. We profile the performance of the different parts of the architecture and our systematic ablations decompose the contributions between the backbone and the self-attention layers.

ACKNOWLEDGMENTS

The authors would like to thank Yufei Li for providing the upstream data assets and model checkpoints used in this work, and for helpful discussions on evaluation methodology.

REFERENCES

- Quentin Berthet, Mathieu Blondel, Olivier Teboul, Marco Cuturi, Jean-Philippe Vert, and Francis Bach. Learning with differentiable perturbed optimizers. In *NeurIPS*, 2020.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv:2401.10774*, 2024.
- Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. End-to-end object detection with transformers. In *ECCV*, 2020.
- Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transport. In *NeurIPS*, 2013.
- Nicola De Cao, Gautier Izacard, Sebastian Riedel, and Fabio Petroni. Autoregressive entity retrieval. In *ICLR*, 2021.
- Paul Dütting, Zhe Feng, Harikrishna Narasimhan, David C. Parkes, and Sai Srivatsa Ravindranath. Optimal auctions through deep learning. In *ICML*, 2019.
- Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed A. Aly, Beidi Chen, and Carole-Jean Wu. Layerskip: Enabling early exit inference and self-speculative decoding. *arXiv:2404.16710*, 2024.
- Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. Mask-predict: Parallel decoding of conditional masked language models. In *EMNLP*, 2019.
- Jiatao Gu, James Bradbury, Caiming Xiong, Victor O.K. Li, and Richard Socher. Non-autoregressive neural machine translation. In *ICLR*, 2018.
- Ruining He and Julian McAuley. Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering. In *Proceedings of the 25th International Conference on World Wide Web*, pp. 507–517, 2016.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv:1503.02531*, 2015.
- Sebastian Hofstätter, Sophia Althammer, Michael Schröder, Mete Sertkan, and Allan Hanbury. Improving efficient neural ranking models with cross-architecture knowledge distillation. *arXiv:2010.02666*, 2020.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv:2106.09685*, 2022.
- Roy Jonker and Anton Volgenant. A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing*, 38:325–340, 1987.
- Harold W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2:83–97, 1955.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *ICML*, 2023.

- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv:2401.15077*, 2024.
- Qi Liu, Kun Ai, Jiaxin Mao, Yanzhao Zhang, Mingxin Li, Dingkun Long, Pengjun Xie, Fengbin Zhu, and Ji-Rong Wen. Diffurank: Effective document reranking with diffusion language models, 2026. URL <https://arxiv.org/abs/2602.12528>.
- Gonzalo Mena, David Belanger, Scott Linderman, and Jasper Snoek. Learning latent permutations with gumbel-sinkhorn networks. In *ICLR*, 2018.
- Felix Petersen, Christian Borgelt, Hilde Kuehne, and Oliver Deussen. Differentiable sorting networks for scalable sorting and ranking supervision. In *ICML*, 2021.
- Ronak Pradeep, Sahel Sharifmoghaddam, and Jimmy Lin. Rankzephyr: Effective and robust zero-shot listwise reranking is a breeze! *arXiv:2312.02724*, 2023.
- Sebastian Prillo and Julian Martin Eisenschlos. Softsort: A continuous relaxation for the argsort operator. In *ICML*, 2020.
- Lihua Qian, Hao Zhou, Yu Bao, Mingxuan Wang, Lin Qiu, Weinan Zhang, Yong Yu, and Lei Li. Glancing transformer for non-autoregressive neural machine translation. In *ACL*, 2021.
- Revanth Gangi Reddy, JaeHyeok Doo, Yifei Xu, Md Arafat Sultan, Deevya Swain, Avirup Sil, and Heng Ji. First: Faster improved listwise reranking with single token decoding. In *EMNLP*, 2024.
- Yuxin Ren, Qiya Yang, Yichun Wu, Wei Xu, Yalong Wang, and Zhiqiang Zhang. Non-autoregressive generative models for reranking recommendation. In *KDD*, 2024.
- Weiwei Sun, Lingyong Yan, Xinyu Ma, Pengjie Ren, Dawei Yin, and Zhaochun Ren. Is chatgpt good at search? investigating large language models as re-ranking agents. *arXiv:2304.09542*, 2023.
- Yi Tay, Vinh Q. Tran, Mostafa Dehghani, Jianmo Ni, Dara Bahri, Harsh Mehta, Zhen Qin, Kai Hui, Zhe Zhao, Jai Gupta, Tal Schuster, William W. Cohen, and Donald Metzler. Transformer memory as a differentiable search index. In *NeurIPS*, 2022.
- Zuowu Zheng, Ze Wang, Fan Yang, Wenqing Ye, Weihua Huang, Wenqiang He, Teng Zhang, and Xingxing Wang. Nga: Non-autoregressive generative auction with global externalities for advertising systems. *arXiv:2506.05685*, 2025.